

8 — API —

— API

###

7 (Promise) , Fetch API . HTTP JSON , async/await . [] → [Fetch API] → [JSON] → [] → [] .

1 Fetch API

```
- **:** , .
- **:** Promise , .then() await .
- **:** fetch('https://api.example.com/data').then(res => res.json())
- **:** 404, 500 HTTP .
- **:** fetch response , (200) response.ok .
```

2 JSON (JavaScript Object Notation)

```
- **:** , - .
- **:** JSON.parse() .
- **:** { "name": "MoAI", "level": 8 }
- **:** JSON ( ) , .
- **:** JSON .json() .
```

3 async/await

```
- **:** (Syntactic Sugar).
- **:** async, await Promise .
- **:** const data = await fetch(url).then(r => r.json());
- **:** await async .
- **:** async Promise .
```

4 HTTP (Methods)

```
- **:** .
- **:** GET(), POST(), PUT(), DELETE() .
- **:** fetch(url, { method: 'POST', body: JSON.stringify(data) })
- **:** (headers) 'Content-Type': 'application/json' .
- **:** Fetch API GET .
```

5 (try/catch)

```
- **:** .
- **:** try catch .
- **:** try { const res = await fetch(url); } catch(err) { console.log(err); }
- **:** fetch , throw .
- **:** catch .
```

6 (Response Handling)

```
- **:**  
- **:** response.status 200(), 404( ) .  
- **:** if (!response.ok) throw new Error(' ');  
- **:**  
- **:** response.ok 200-299 true .
```

###

```
| | fetch | XMLHttpRequest ( ) |  
| :--- | :--- | :--- |  
| | (Promise ) | ( ) |  
| | , | (Low-level) |
```

```
| | JSON ( ) | XML, Text |  
| | REST API | |
```

###

1. **:** API fetch .
2. **:** POST .
3. **:** fetch , ' ' .

TOP 6

1. .json() await (Promise)
2. async await
3. response.ok (404)
4. POST headers JSON
5. fetch
6. JSON ()

&

```
- **:** '(Fetch) (Promise) , response.ok!'  
- **:** JSON.parse() .json() ( vs )
```

6

- [] fetch ?
- [] JSON ?
- [] async/await ?
- [] HTTP ?
- [] POST ?
- [] try/catch ?

1. Fetch API , (200 OK) ?

A. response.status == 200

B. response.ok

C. response.success

D. response.json()

: B

: Fetch API

: response.ok HTTP 200-299 true .

2. JSON ?

A. JSON .

B. (Key) .

C. JSON (//) .

D. .

: C

: JSON

: JSON , .

3. async/await ?

A. await async .

B. async Promise .

C. try/catch .

D. await (,) .

: B

: async/await

: async Promise .

4. HTTP ?

A. GET

B. POST

C. FETCH

D. READ

: B

: HTTP

: POST .

5. Fetch API ?

A. const data = await fetch(url);

B. const data = fetch(url).json();

C. const res = await fetch(url); const data = await res.json();

D. const data = parse(fetch(url));

: C

: Fetch API

: fetch , .json() Promise await .

:

API(: JSONPlaceholder) .

[]

- fetch async/await
- (response.ok)
-
- try/catch

[] fetch, async, json, try-catch

[]

1. HTML
2. async fetch
3. li

[] (40) + (30) + (30)

[] 100